

PROJE 01 / 20 · ÇALIŞAN TEMEL SÜRÜM

Abonelik ve Fatura Takip Paneli

Müşteri aboneliklerini ve dönem faturalarını yönet; tekrar ödeme olaylarını güvenle işle.

Ödeme olayları simüle edilir

Bu proje nedir?

Frontend kullanıcının gördüğü ekranları ve etkileşimleri, backend iş kurallarını ve veri erişimini yürütür. Bu uygulama ikisini aynı origin üzerinde birleştiren bağımsız bir full-stack örnektir. Üretim ürünü değil, üzerinde geliştirme yapabileceğin çalışan başlangıçtır.

DYFULL-STACK LAB / 01 / 20Deniz DemoÇıkış

PORTFOLYO PROJESİ · ÇALIŞAN TEMEL SÜRÜM

Abonelik ve Fatura Takip Paneli

Müşteri aboneliklerini ve dönem faturalarını yönet; tekrar ödeme olaylarını güvenle işle.

Ödeme olayları simüle edilir

İşlem tamamlandı

AYLIK TEKRARLAYAN GELİR

₺990,00

1 aktif abonelik · yıllık plan / 12

Abonelik oluştur

Müşteri

Örnek Teknoloji

Tutar (TL)

990

Dönem

Aylık

Ekle

Gerçek uygulama ekranı · yerel demo verileri

Teslim edilen kapsam

Aylık/yıllık abonelik ekleme · Aktif-pasif abonelik durumu · Döneme göre fatura oluşturma · Tekilleştirilmiş ödeme olayı · Kuruş bazlı MRR hesabı

Bu sürümde olmayanlar

Gerçek ödeme sağlayıcısı, e-fatura, iade ve vergi hesaplaması yok. Müşteri ayrı tablo yerine abonelik alanıdır. Yıllık plan faturası üretiminde dönem seçimini kullanıcı yapar; otomatik takvim işi yok.

02 / Backend ve veri modeli

Para tamsayı kuruş olarak tutulur. Aktif yıllık bedel 12'ye bölünür, aylık bedelle toplanır ve sonuç sunum için yuvarlanır. subscriptions birden fazla invoices kaydına sahiptir. UNIQUE(subscription_id,period) aynı dönem faturasını engeller. events.event_id tekrarları ayıklar; olay kaydı ve paid geçişi tek transaction içindedir.

Gerçek SQL şeması

```
CREATE TABLE subscriptions(id TEXT PRIMARY KEY,owner TEXT NOT NULL REFERENCES
users(id),customer TEXT NOT NULL,amount INTEGER NOT NULL CHECK(amount>0),interval TEXT
NOT NULL CHECK(interval IN ('month','year')),active INTEGER NOT NULL DEFAULT 1);
```

```
CREATE TABLE invoices(id TEXT PRIMARY KEY,subscription_id TEXT REFERENCES
subscriptions(id),period TEXT NOT NULL,amount INTEGER NOT NULL,status TEXT NOT NULL
DEFAULT 'open',UNIQUE(subscription_id,period));
```

```
CREATE TABLE events(event_id TEXT PRIMARY KEY,invoice_id TEXT REFERENCES
invoices(id),created_at TEXT NOT NULL);
```

Tam şema: schema.sql. users ve sessions ortak kimlik tablolarıdır. owner/user alanları kullanıcıyı, PRIMARY KEY kaydı, UNIQUE tekiliği ve REFERENCES ilişkisi belirtir. Para kullanılan projelerde INTEGER kuruş, zaman alanlarında ISO metin veya epoch kullanılır.

Kod okuma sırası

Dosya	Ne arayacaksın?
client/Workspace.vue	Eylemi tetikleyen form ve API isteği
client/api.ts	JSON isteği, hata yakalama ve mesaj
server/project.js	Yetki kontrolü → kural → SQL → yanıt
server/core.js	Parametrelili SQL, transaction, oturum, SSE

Neden bu mimari?

Aynı dilde frontend/backend öğrenmeyi kolaylaştırır. Node.js 24 ve Express HTTP katmanıdır; Vue arayüzü yönetir. SQLite ayrıca sunucu kurmadan kalıcılık sağlar. Senkron veritabanı işlemleri küçük yerel demo için seçildi; çok süreçli, yüksek trafikli üretim mimarisi iddiası yok.

03 / API sözleşmesi

Aşağıdaki yollar doğrudan teslim edilen server/project.js dosyasından çıkarılmıştır. JSON alan adları kodla aynıdır. :id ve benzeri path alanları oluşturma yanıtından alınmalıdır. Liste/görünüm uçlarında 200, yeni kayıt uçlarında 201, kuyruk kabulünde 202 kullanılır.

Yöntem / yol	Girdi alanları	Başarı
GET /api/subscriptions	Gövde yok	200
POST /api/subscriptions	JSON: customer, amount, interval	201
PATCH /api/subscriptions/:id	JSON: active	200
POST /api/invoices	JSON: subscriptionId, period	201
GET /api/invoices	Gövde yok	200
POST /api/payment-events	JSON: invoiceId, eventId	200
GET /api/summary	Gövde yok	200

Somut istek örneği

```
POST /api/subscriptions
{"customer": "Acme", "amount": 120000, "interval": "year"}
201 {"id": "<subscription-id>"}
```

Oturum ve hata biçimi

POST /api/auth/login: email ve password. Başarılı giriş HttpOnly çerez verir. GET /api/auth/me kullanıcıyı, POST /api/auth/logout çıkışı yönetir. Tarayıcı aynı origin kullanır. Genel hata biçimi: {"error": "açıklama"}. Yetkisiz alan işlemleri sunucuda engellenir; frontend kontrolü tek başına yeterli değildir.

Kod	Anlam
401 / 403	Giriş gerekli / rol veya origin reddedildi
404 / 409	Kayıt görünmüyor / durum veya benzersizlik çakışması
422 / 429	Geçersiz girdi / giriş veya uygulama kotası
500 / 502 / 503	Beklenmeyen hata / dış servis hatası / kapalı veya ayarsız servis

Tüm uçlar bütün kodları üretmez. İstek/yanıt gövdelerini tests/project.test.js ve scripts/seed.js içindeki çalışan örneklerle karşılaştır.

04 / Test ve kabul kriterleri

Bir projenin çalışması yalnız ekrana veri gelmesi değildir. Yetki, tekrar istek, yanlış girdi ve yarış koşulları da beklenen sonucu üretmelidir. Aşağıdaki maddeler teslim edilen temel sürümün kontrol listesidir.

- Aynı ödeme olayı ikinci kez geldiğinde applied=false dönmeli.
- Yıllık 120.000 kuruş için MRR 10.000 kuruş olmalı.
- Başka hesabın faturasına ödeme olayı gönderilememeli.
- Aynı abonelik/dönem için ikinci fatura engellenmeli.

Otomatik test nasıl çalışır?

npm test, Node test runner ile gerçek HTTP sunucusunu rastgele yerel portta başlatır. İki kullanıcıyla oturum açar, uç noktalara istek gönderir ve durum/yanıtları assert ile denetler. Veritabanı :memory: olduğu için kendi data/app.sqlite dosyan etkilenmez. Projeye özgü testler tests/project.test.js içindedir.

```
npm ci
npm run typecheck
npm test
npm run build
```

Test kodundan örnek

```
const s=await
post('/subscriptions',{customer:'Acme',amount:120000,interval:'year'},201);
await post('/invoices',{subscriptionId:s.id,period:'2026-09'},201);const inv=(await
get('/invoices'))[0];
assert.equal((await post('/payment-
events',{invoiceId:inv.id,eventId:'event-1'})).applied,true);
assert.equal((await post('/payment-
events',{invoiceId:inv.id,eventId:'event-1'})).applied,false);
assert.equal((await get('/summary')).mrr,10000);
```

Tarayıcıda kontrol et

Bir başarılı işlem, bir hatalı girdi ve boş durum göster. 390 px mobil görünümde yatay taşmayı kontrol et. Sayfayı yenilediğinde kalıcı kayıtlar durmalı. Network panelinde isteğin gövdesini ve dönen HTTP kodunu oku; başarısız işlemi başarılıymış gibi göstermemelisin.

Doğrulamanın sınırı

API entegrasyon testleri, üretim derlemesi ve Vue tip/şablon kontrolü uygulandı. Bunlar yük testi, penetrasyon testi veya tüm tarayıcılarda kapsamlı uçtan uca test yerine geçmez.

05 / Çalışma ve GitHub planı

İlk günün somut hedefi

Abonelik formundan kayıt oluştur; veritabanında amount alanını izle. Aylık tutarı kullanıcıya TL, API'ye kuruş olarak göndermeyi kendin değiştirip dene.

Çalışma bloğu	Teslim edeceğin çıktı
1 · Kurulum / 30-60 dk	Çalışan ekran, örnek veri ve ilk başarılı API isteği
2 · Kod okuma / 45-90 dk	Formdan SQL sorgusuna kadar bir isteğin açıklaması
3 · Geliştirme / 60-120 dk	Bir iş kuralı veya arayüz davranışında kendi değişikliğin
4 · Doğrulama / 30-60 dk	İlgili test ve README değişikliği; yeni ekran görüntüsü

Süreler tahmindir; hata çözme ve yeni kavramlar daha uzun sürebilir. Bu plan hazır temel sürüm üzerinde çalışmak içindir. Projeyi sıfırdan ve üretim kalitesinde bir günde bitirme vaadi değildir.

İleri sürüm: dört gerçek geliştirme

- 1. Plan ve müşteri tablolarını ayır
- 2. Dönem üreticisi ve otomatik fatura işi ekle
- 3. Test sağlayıcısı webhook imzasını doğrula
- 4. Owner/viewer yetkilerini ekle

Anlamalı commit sırası

```
git add .
git commit -m "chore: add subscription-billing baseline"
# Kendi değişikliğini yaptıktan sonra:
git add client server tests README.md
git commit -m "feat: describe your actual improvement"
```

Yeni boş GitHub reposuna remote ekleyip push et. Burada gerçek hesap/repo adresi bulunmadığı için otomatik yükleme yapılmadı. node_modules, data ve .env dosyalarını hariç tut. Her gün bir gerçek ilerleme paylaş; commit sayısını artırmak için geçmiş üretme.

Mülakatta anlat

Olayın iki kez gelmesiyle iki farklı olayın aynı faturaya gelmesi arasındaki fark ne? Otomatik fatura dönemini nasıl tekilleştirirsin?

Özel kurulum / ilerleme notu

Önce yerel sürümü tamamla. Sonraki aşamada PostgreSQL, migration, pagination ve gerçek kullanıcı yönetimini ihtiyaca göre ekle. Ortak kurulum ayrıntıları README.md ve başlangıç rehberindedir.